

第三章 语义分析

冯洋
南京大学计算机学院
fengyang@nju.edu.cn



提纲

- 语法制导定义的基本概念
- 抽象语法树与类型
- 语法制导翻译的应用
- 工业级编译器中的语义分析

语法制导定义的基本概念

- 计算机程序是由很多语法结构构成
- 相同的语法单元在不同的语法结构中可能包含不同的语义信息
- 使用上下文无关文法引导语言的翻译
 - CFG的非终结符号代表了语言的某个构造
 - 程序设计语言的构造由更小的构造组合而成
 - 一个构造的语义可以由小构造的含义**综合**而来
 - 比如：表达式 $x+y$ 的类型由 x 、 y 的类型和运算符 $+$ 决定
 - 也可以从附近的构造**继承**而来
 - 比如：声明`int x;`中 x 的类型由它左边的类型表达式决定

语法制导定义的基本概念

- 背景：
 - 上下文无关文法只能描述语法
 - 需要结合语义规则来完成语义分析与中间代码生成
- 语法制导的定义 (SDD: Syntax-Directed Definition):
 - 一种形式系统，将语义规则附着到文法产生式上
 - 将文法符号和某些属性相关联
 - 并通过**语义规则**来描述如何计算属性的值

语法树 + 属性计算 = 语义分析

语法制导定义的基本概念

- 语法制导定义(SDD: Syntax-Directed Definition)
 - 将文法符号和某些属性相关联
 - 并通过**语义规则**来描述如何计算属性的值
 - $E \rightarrow E_1 + T$ $E.code = E_1.code || T.code || '+'$
 - 属性`code`代表中缀表达式的逆波兰表示(后缀表示法)，规则说明加法表达式的逆波兰表示由两个分量的逆波兰表示并置，然后加上 $+$ 得到
- 语法制导翻译(SDD: Syntax-Directed Translation)
 - 在产生式体中加入语义动作，并在适当的时候执行这些语义动作
 - $E \rightarrow E_1 + T$ $\{ \text{print '+'} \}$

语法制导定义的基本概念

特性	SDD (属性定义)	SDT (语义动作)
表达方式	抽象、数学化	具体、嵌入代码
可读性	高, 便于教学与证明	偏向实现, 代码细节多
应用	理论建模, 描述属性依赖	编译器工具 (如 YACC/Bison)

语法制导定义的基本概念

- 什么是逆波兰表达式?

语法制导定义的基本概念

- 什么是逆波兰表达式?
- 编译器的中间代码生成主要依赖**栈式**计算模型 (例如早期的 JVM、栈式汇编)
- 经典的算术表达式 (如 $a + b * c$) 用中缀形式不利于机器直接执行
- 如何处理:
 - 转换成逆波兰表达式 (RPN, Reverse Polish notation) 或后缀表达式 (如 $a b c * +$), 就可以直接利用栈来计算

语法制导定义的基本概念

- 编译器的中间代码生成主要依赖**栈式**计算模型 (例如早期的 JVM、栈式汇编)
- 经典的算术表达式 (如 $a + b * c$) 用中缀形式不利于机器直接执行
- 如何处理:
 - 转换成逆波兰表达式 (RPN, Reverse Polish Notation) 或后缀表达式 (如 $a b c * +$), 就可以直接利用栈来计算
 - 波兰的数理科学家 Jan Łukasiewicz 发明
 - 中缀表达式: $(1 + 2) * (3 + 4)$ 加减乘除等运算符写在中间
 - 波兰表达式: $(* (+ 1 2) (+ 3 4))$, 将运算符写在前面, 因而也称为前缀表达式。
 - 逆波兰表达式: $((1 2 +) (3 4 +) *)$, 将运算符写在后面, 因而也称为后缀表达式

语法制导定义的基本概念

- SDD与RPN示例

文法定义如下:	文法定义如下:
$E \rightarrow E + T$	$E \rightarrow E1 + T \quad \{ E.postfix = E1.postfix \parallel T.postfix \parallel "+" \}$
$E \rightarrow E - T$	$E \rightarrow E1 - T \quad \{ E.postfix = E1.postfix \parallel T.postfix \parallel "-" \}$
$E \rightarrow T$	$E \rightarrow T \quad \{ E.postfix = T.postfix \}$
$T \rightarrow T * F$	$T \rightarrow T1 * F \quad \{ T.postfix = T1.postfix \parallel F.postfix \parallel "*" \}$
$T \rightarrow T / F$	$T \rightarrow T1 / F \quad \{ T.postfix = T1.postfix \parallel F.postfix \parallel "/" \}$
$T \rightarrow F$	$T \rightarrow F \quad \{ T.postfix = F.postfix \}$
$F \rightarrow (E)$	$F \rightarrow (E) \quad \{ F.postfix = E.postfix \}$
$F \rightarrow id$	$F \rightarrow id \quad \{ F.postfix = id.name \}$

语法制导定义的基本概念

- SDD与RPN示例

文法定义如下:
$E \rightarrow E1 + T \quad \{ E.postfix = E1.postfix \parallel T.postfix \parallel "+" \}$
$E \rightarrow E1 - T \quad \{ E.postfix = E1.postfix \parallel T.postfix \parallel "-" \}$
$E \rightarrow T \quad \{ E.postfix = T.postfix \}$
$T \rightarrow T1 * F \quad \{ T.postfix = T1.postfix \parallel F.postfix \parallel "*" \}$
$T \rightarrow T1 / F \quad \{ T.postfix = T1.postfix \parallel F.postfix \parallel "/" \}$
$T \rightarrow F \quad \{ T.postfix = F.postfix \}$
$F \rightarrow (E) \quad \{ F.postfix = E.postfix \}$
$F \rightarrow id \quad \{ F.postfix = id.name \}$

$a + b * c$

推导过程:

$b * c \rightarrow "b c *"$

$a + (b c *) \rightarrow "a b c * +"$

语法制导定义的基本概念

- SDD是对上下文无关文法的推广
 - SDD = 上下文无关文法 + 属性 + 语义规则
 - 将每个文法符号和一个语义属性集合相关联
 - 将每个产生式 and 一组语义规则相关联，这些规则用于计算该产生式中各文法符号的属性值
- 为什么需要属性？
 - 语法分析只能保证程序符号串符合文法规则
 - 但程序的语义信息（如类型、值、作用域等）需要额外描述
 - 属性 (Attribute) 就是为了解决这个问题
 - 给文法符号附加额外信息
 - 在分析过程中逐步计算并传播这些信息

语法制导定义的基本概念

- 一个例子：假设我们需要知道一个表达式的类型，以及对应代码将它的值存放在何处，我们就需要两个属性：type, place
- 产生式规则： $E \rightarrow E_1 + T$
- 语义规则：(假设只有int/float类型)
 - $E.type = \text{if } (E_1.type == T.type) \ T.type \ \text{else float}$
 - $E.place = \text{newTempPlace}();$ //返回一个新的内存位置；
- 产生式规则： $F \rightarrow id$
 - $F.type = \text{lookupIDTable}(id.lexValue) \rightarrow type;$
 - $F.place = \text{lookupIDTable}(id.lexValue) \rightarrow address;$

语法制导定义的基本概念

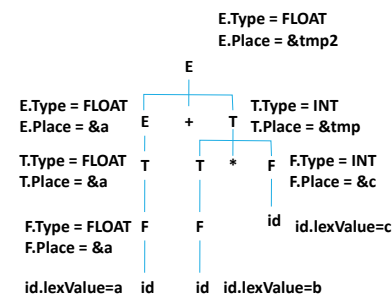
- 属性 (Attribute): 与文法符号相关联的值，用来存储语义信息
- 在属性文法 (Attributed Grammar) 中，属性之间可能存在依赖关系

$$E.val = E1.val + T.val$$

语法制导定义的基本概念

- SDD是上下文无关文法和属性/规则的结合
 - 属性和文法符号相关联，按照需要来确定各个文法符号需要哪些属性
 - 目的：通过在文法的每个产生式上附加“语义规则”，把语法结构和语义计算联系起来
 - 规则和产生式相关联
 - 主要就是为CFG中的文法符号设置语义属性，具体来说，主要就是为一些语义属性附加到代表语言构造的文法符号上，从而把信息和语言构造联系起来
- 对于文法符号X和属性a，用X.a表示分析树中的某个标号为X的结点的值
- 一个分析树结点和它的分支对应于一个产生式规则，而对应的语义规则确定了这些结点上的属性的取值

语法制导定义的基本概念



假设a, b, c是已经声明的全局变量，a的类型为FLOAT，b, c的类型为INT

中间未标明的T和F的type和address都是INT和&b；

a+b*c的语法分析树以及属性值

语法制导定义的基本概念

- 属性 (Attribute): 与文法符号相关联的值，用来存储语义信息
- 在属性文法 (Attributed Grammar) 中，属性之间可能存在依赖关系

$$E.val = E1.val + T.val$$

这里 E.val 的计算依赖于 E1.val 和 T.val

问题：我们如何确定属性计算的顺序？

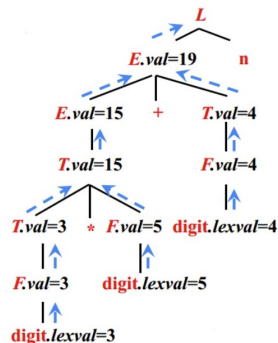
引入依赖关系的图形表示来形式化表达属性之间的依赖

语法制导定义的基本概念

- 实践中很少先构造语法分析树再进行SDD求值
- 但在分析树上求值有助于翻译方案的可视化，便于理解
- 注释语法分析树
 - 包含了各个结点的各属性值的语法分析树
- 步骤：
 - 对于任意的输入串，首先构造出相应的分析树。
 - 给各个结点（根据其文法符号）加上相应的属性值
 - 按照语义规则计算这些属性值即可

语法制导定义的基本概念

产生式	语义规则
1) $L \rightarrow E \ n$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



- 对于输入3*5+4n
- n表示结束

语法制导定义的基本概念

- 综合属性示例：
 - 文法： $E \rightarrow E + T \mid T$ $T \rightarrow \text{num}$
 - $E \rightarrow E1 + T$ { $E.val = E1.val + T.val$ }
 - $E \rightarrow T$ { $E.val = T.val$ }
 - $T \rightarrow \text{num}$ { $T.val = \text{num.lexval}$ }

val 是 **综合属性**，计算方向是 **从下到上**

语法制导定义的基本概念

- 按照分析树中的分支对应的文法产生式，应用相应的语义规则计算属性值
- 计算顺序问题：
 - 如果某个结点N的属性a为 $f(N_1.b_1, N_2.b_2, \dots, N_k.b_k)$ ，那么我们需要先算出 $N_1.b_1, N_2.b_2, \dots, N_k.b_k$ 的值。
- 如果我们可以给各个属性值排出计算顺序，那么这个注释分析树就可以计算得到
 - S属性的SDD一定可以按照自底向上的方式求值
- 下面的SDD不能计算
 - $A \rightarrow B$ $A.s = B.i$; $B.i = A.s + 1$;

语法制导定义的基本概念

- 分为两类：
 - 综合属性 (Synthesized Attribute): 自下而上计算，从子结点传到父结点，常用于表达式的值、类型计算
 - 继承属性 (Inherited Attribute): 自上而下或兄弟结点之间传递，常用于符号表传递、上下文约束、类型环境
- 不允许结点的继承属性通过子结点上的属性来定义，但是允许结点的综合属性依赖于结点本身的继承属性
- 终结符号有综合属性（由词法分析获得），但是没有继承属性

语法制导定义的基本概念

- 继承属性示例：
 - 文法： $L \rightarrow T L'$ $L' \rightarrow , T L' \mid \epsilon$ $T \rightarrow \text{id}$
 - $T.type$ 由父节点传递
 - $\text{id.entry.type} = T.type$ （把类型传给符号表项）
 - $T.type$ 就是 继承属性，它随文法结构向下传递

语法制导定义的基本概念

- 目标：计算表达式行L的值（属性val）
- 计算L的val值需要E的val值
- E的val值又依赖于E和T的val值
- ...
- 终结符号digit有综合属性lexval

产生式	语义规则
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

属性文法基本概念

- Recap：左递归的消除
- 问题：SDD的左递归如何消除？

产生式	PRODUCTION
1) $L \rightarrow E \mathbf{n}$	1) $T \rightarrow F T'$
2) $E \rightarrow E_1 + T$	2) $T' \rightarrow * F T'_1$
3) $E \rightarrow T$	3) $T' \rightarrow \epsilon$
4) $T \rightarrow T_1 * F$	4) $F \rightarrow \mathbf{digit}$
5) $T \rightarrow F$	
6) $F \rightarrow (E)$	
7) $F \rightarrow \mathbf{digit}$	



属性文法基本概念

- Recap：左递归的消除

$E \rightarrow E_1 + T \quad \{ E.val = E_1.val + T.val \}$
 $E \rightarrow T \quad \{ E.val = T.val \}$

属性文法基本概念

- Recap：左递归的消除

$E \rightarrow E_1 + T \quad \{ E.val = E_1.val + T.val \}$
 $E \rightarrow T \quad \{ E.val = T.val \}$

- 消除左递归的过程中，如何处理语义规则？

属性文法基本概念

- 消除左递归的语义规则改写思路

$A \rightarrow A \alpha \mid \beta$
 $A.val = f(A_1.val, \alpha.val) \quad // A \rightarrow A \alpha$
 $A.val = g(\beta.val) \quad // A \rightarrow \beta$

- 消除左递归后

$A \rightarrow \beta A'$
 $A' \rightarrow \alpha A' \mid \epsilon$

- 语义规则的修改套路（公式化方式）

➤在消除左递归时：

- 给A'引入一个继承属性 inh，用于保存已经累积的结果
- 给A'定义一个综合属性 syn，用于将最终值返回给A

$A \rightarrow \beta A' \quad \{ A'.inh = g(\beta.val); A.val = A'.syn \}$
 $A' \rightarrow \alpha A_1' \quad \{ A_1'.inh = f(A'.inh, \alpha.val); A'.syn = A_1'.syn \}$
 $A' \rightarrow \epsilon \quad \{ A'.syn = A'.inh \}$

属性文法基本概念

- 语义规则的修改套路（公式化方式）

➤在消除左递归时：

- 给A'引入一个继承属性 inh，用于保存已经累积的结果
- 给A'定义一个综合属性 syn，用于将最终值返回给A

$A \rightarrow \beta A' \quad \{ A'.inh = g(\beta.val); A.val = A'.syn \}$
 $A' \rightarrow \alpha A_1' \quad \{ A_1'.inh = f(A'.inh, \alpha.val); A'.syn = A_1'.syn \}$
 $A' \rightarrow \epsilon \quad \{ A'.syn = A'.inh \}$

➤基本修改逻辑

- 原始逻辑：每次用 $A \rightarrow A \alpha$ ，把已有的结果与新部分结合。
- 新逻辑：
 - A' 用 inh 保存“已有的结果”。
 - 每次扩展 $A' \rightarrow \alpha A'$ ，就把 inh 与 α 结合，传给下一个。
 - 当 $A' \rightarrow \epsilon$ 时，说明结束， $syn = inh$ 。
- 保证等价：最终A.val与原始定义完全一致。

属性文法基本概念

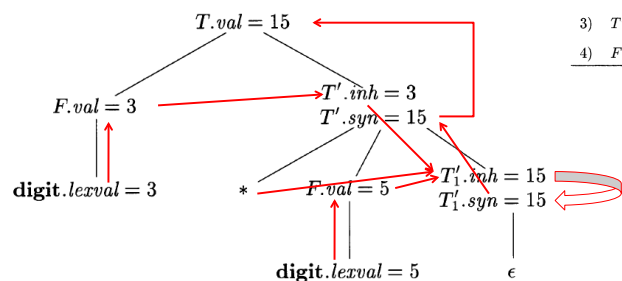
- 语义规则的修改套路（公式化方式）
 - 引入继承属性，保存之前的计算结果
 - 在改写后文法中，利用辅助非终结符 E' 进行迭代的累积
 - 假设有一个左递归的文法： $E \rightarrow E+T \mid T$
 - 对应的语义规则可能是： $E \rightarrow E1+T: E.val=f(E1.val,T.val)$
 - 其中： $E \rightarrow T: E.val=g(T.val)$
 - f 表示如何把 左边的结果 与 当前的项组合
 - g 表示如何把 基本的 T 转换为一个初始的 E

属性文法基本概念

- 消除左递归后的语义规则改写思路
 - 引入继承属性，保存之前的计算结果
 - 在改写后文法中，利用辅助非终结符 E' 进行迭代的累积
- $E \rightarrow E1+T \quad \{ E.val = E1.val + T.val \}$
 $E \rightarrow T \quad \{ E.val = T.val \}$
- $E \rightarrow T E' \quad \{ E'.inh = T.val; E.val = E'.syn \}$
 $E' \rightarrow + T E1' \quad \{ E1'.inh = E'.inh + T.val; E'.syn = E1'.syn \}$
 $E' \rightarrow \epsilon \quad \{ E'.syn = E'.inh \}$
- $E' \rightarrow E+T$: 每次遇到 $+T$, 就把 $T.val$ 加到 $E.val$ 中
 E' 通过 继承属性 inh 保存前面累积的部分。
 每次扩展 $E' \rightarrow +T E'$ 时:
 取出 $E'.inh$ (已有和式), 加上新的 $T.val$, 传给下一个 E' 的 inh 。
 最终 $E'.syn$ 就是完整的累加结果, 赋值给 $E.val$ 。
 E' 承担了“重复相加”的作用
- $E'.inh$ (继承属性): 保存当前计算的部分结果。
 $E'.syn$ (综合属性): 返回最终结果。

语法制导定义的基本概念

- 适用于自顶向下分析的SDD



产生式	语义规则
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

属性文法基本概念

- 语义规则的修改套路（公式化方式）
 - 引入继承属性，保存之前的计算结果
 - 在改写后文法中，利用辅助非终结符 E' 进行迭代的累积
 - 假设有一个左递归的文法： $E \rightarrow E+T \mid T$
 - 对应的语义规则可能是： $E \rightarrow E1+T: E.val=f(E1.val,T.val)$
 - 其中： $E \rightarrow T: E.val=g(T.val)$
 - f 表示如何把 左边的结果 与 当前的项组合
 - g 表示如何把 基本的 T 转换为一个初始的 E
- $\bullet g(x)$: 初始化函数，把一个基本值 $T.val$ 转换成表达式的初始结果。在算术表达式例子里： $g(T.val)=T.val$, 因为单个 T 就是初始表达式。
- $\bullet f(x, y)$: 组合函数，把已有的部分结果 x 和新解析出来的 $T.val=y$ 按照操作符结合起来。在例子里： $f(x,y)=x+y$ 。

属性文法基本概念

- Recap: 左递归的消除
- $E \rightarrow E1+T \quad \{ E.val = E1.val + T.val \}$
 $E \rightarrow T \quad \{ E.val = T.val \}$
- 消除左递归后的语义规则改写思路
- $E \rightarrow E1+T \quad \{ E.val = E1.val + T.val \}$
 $E \rightarrow T \quad \{ E.val = T.val \}$
- $E \rightarrow T E' \quad \{ E'.inh = T.val; E.val = E'.syn \}$
 $E' \rightarrow + T E1' \quad \{ E1'.inh = E'.inh + T.val; E'.syn = E1'.syn \}$
 $E' \rightarrow \epsilon \quad \{ E'.syn = E'.inh \}$
- 为什么语义传递不变?
 关键是 逻辑等价性:
 原始递归式:
 $E.val = (...((T1.val + T2.val) + T3.val) ...)$
 新文法式:
 $E'.inh$ “携带” $(...((T1.val + T2.val) + T3.val) ...)$
 虽然语法树结构不同, 计算过程本质相同。
 最终 $E.val$ 与原始定义保持一致。
- $E'.inh$ (继承属性): 保存当前计算的部分结果。
 $E'.syn$ (综合属性): 返回最终结果。

语法制导定义的基本概念

- SDD中属性的求值顺序
 - 在对SDD的求值过程中，如果结点N的属性a依赖于结点 M_1 的属性 a_1 , M_2 的属性 a_2 , ...。那么我们必须先计算出 M_1 的属性，才能计算N的属性a
 - 使用依赖图来表示计算顺序
 - 显然，这些值的计算顺序应该形成一个偏序关系。如果依赖图中出现了环，表示属性值无法计算

语法制导定义的基本概念

- 依赖的图形化表示：**依赖图**
- 描述了某棵特定的分析树上各个属性实例之间的信息流（计算顺序）
 - 从实例 a_1 到实例 a_2 的有向边表示计算 a_2 时需要 a_1 的值（必须先计算 a_1 ，再计算 a_2 ）
- 对于标号为X的分析树结点N，和X关联的每个属性a都对应依赖图的一个结点N.a
- 结点N对应的产生式的语义规则通过X.c计算了A.b的值，且在分析树中X和A分别对应于 N_1 和 N_2 ，那么从 $N_1.c$ 到 $N_2.b$ 有一条边

语法制导定义的基本概念

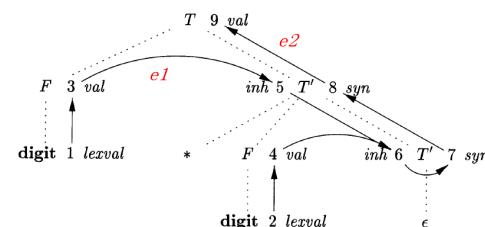
- 各个属性的值需要按照依赖图的拓扑顺序计算
 - 如果依赖图中存在环，则属性计算无法进行
- 给定一个SDD，很难判定是否存在一棵分析树，其对应的依赖图包含环
- 怎样的SDD可以和以前所介绍的语法分析结合在一起？

语法制导定义的基本概念

- 只包含综合属性的SDD称为S属性的SDD
 - 每个语义规则都根据产生式体中的属性值来计算头部非终结符号的属性值
- **S属性的SDD可以和LR语法分析器一起实现**
 - 栈中的状态可以附加相应的属性值
 - 在进行归约时，按照语义规则计算归约得到的符号的属性值
- 语义规则不应该有复杂的副作用
 - 要求副作用不影响其它属性的求值
 - 没有副作用的SDD称为属性文法

语法制导定义的基本概念

- $3*2$ 的注释分析树；
- $T \rightarrow FT' \{T.val = T'.syn; T'.inh = F.val;\}$
 - 边 $e1$ 、 $e2$
- 可能的计算顺序：
 - 1,2,3,4,5,6,7,8,9



产生式	语义规则
1) $T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * FT'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

语法制导定义的基本概念

- 特定类型的SDD一定不包含环，且有固定排序模式
 - S属性的SDD
 - L属性的SDD
- 对于这些类型的SDD，我们可以确定属性的计算顺序，且可以把不需要的属性（及分析树结点）抛弃以提高效率
- 这两类SDD可以很好地和我们已经研究过的语法分析相结合

语法制导定义的基本概念

- L属性的SDD，其中，每个属性
 - 要么是综合属性
 - 要么是继承属性，且产生式 $A \rightarrow X_1X_2 \cdots X_n$ 中计算 $X_i.a$ 的规则只能使用
 - A的继承属性
 - X_i 左边的文法符号 X_j 的继承属性或综合属性
 - X_i 自身的继承或综合属性，且这些属性之间的依赖关系不形成环

语法制导定义的基本概念

- S属性文法（只有综合属性）一定不会有环（因为信息只往上走）
- L属性文法（继承 + 综合，受限条件）也能保证不存在环路（因为继承只依赖父或左兄弟）
- 属性的继承/综合划分，帮助我们构造 S 属性文法与 L 属性文法，这两类文法天然避免了依赖图中的环

S文法与L文法是对属性依赖的限制，它们是“子集”，设计上就是为了 保证可行的无环计算顺序

语法制导定义的基本概念

- 按照后序遍历的顺序计算属性值即可

```
postorder (N)
{
    for (从左边开始，对N的每个子结点C)
        postorder (C);
    //递归调用返回时，各子结点的属性计算完毕
    对N的各个属性求值；
}
```

- 在LR分析过程中，我们实际上不需要构造分析树的结点

语法制导定义的基本概念

- 语义规则（Semantic Rules）
 - 定义：语义规则是抽象层次上的定义，用于描述属性如何从文法符号传播、计算和约束；它通常写在属性文法（Attribute Grammar, AG）的形式化体系中
- 语义动作（Semantic Actions）
 - 定义：语义动作是实现层次上的指令，通常嵌入在文法产生式中的可执行代码，在语法分析过程中被触发执行

语义规则：逻辑层面的“应当如何计算属性”
语义动作：实现层面的“在编译器里怎么做”

语义规则 = 数学公式
语义动作 = 代码实现

语法制导定义的基本概念

- S属性的SDD: 每个属性都是综合属性
- 都是根据子构造的属性计算出父构造的属性
- 在依赖图中，总是通过子结点的属性值来计算父结点的属性值。可以和自顶向下、自底向上的语法分析过程一起计算
 - 自底向上
 - 在构造分析树的结点的同时计算相关的属性（此时其子结点的属性必然已经计算完毕）
 - 自顶向下
 - 递归下降分析中，可以在过程A()的最后计算A的属性（此时A调用的其他过程（对应于子结构）已经调用完毕）

语法制导定义的基本概念

- L属性的SDD: 每个属性
 - 要么是综合属性
 - 要么是继承属性，且产生式 $A \rightarrow X_1 X_2 \dots X_n$ 中计算 X_i 的规则只能使用
 - A的继承属性
 - X_i 左边的文法符号 X_j 的继承属性或综合属性
 - X_i 自身的继承或综合属性，且这些属性之间的依赖关系不形成环
- 特点
 - 依赖图的边
 - 继承属性从左到右，从上到下
 - 综合属性从下到上
 - 在扫描过程中，计算一个属性值时，和它相关的依赖属性都已经计算完毕

语法制导定义的基本概念

- 副作用（Side effect）：语义规则执行时对外部运行时或全局状态的修改，例如：
 - 修改符号表（insert、update）
 - 生成/输出中间代码或目标代码（emit）
 - 写日志/报错（print/error）
 - 修改全局计数器、序号等
- 核心问题：副作用如果位置不当或在不确定的求值顺序下执行，会导致不一致或难以重现的语义错误。

语法制导定义的基本概念

- 具有受控副作用的语义规则
- 属性文法没有副作用，但增加了描述的复杂度
 - 比如语法分析时如果没有副作用，标识符表就必须作为属性传递
 - 可以把标识符表作为全局变量，然后通过副作用函数来添加新标识符

语法制导定义的基本概念

- 具有受控副作用的语义规则
- 属性文法没有副作用，但增加了描述的复杂度
 - 比如语法分析时如果没有副作用，标识符表就必须作为属性传递
 - 可以把标识符表作为全局变量，然后通过副作用函数来添加新标识符
- 受控的副作用需要满足的条件：
 - 先行条件已满足：动作执行时，它所依赖的所有属性/信息均已被正确计算。
 - 执行顺序确定：副作用的执行位置在语法/语义规则中明确，与解析策略（自顶向下或自底向上）一致。
 - 不会改变依赖图的合法性（不会引入新的循环依赖或破坏属性求值结果）。
- 若任一条件不满足，副作用就不受控，可能引发错乱（例如重复插入、提前生成错误的代码）

语法制导定义的基本概念

- 为什么不能随意放置副作用？

```
D → T id
// 意图：把 id 加入符号表，类型就是 T.type
动作： addType(id.name, T.type)
```

- 合法情形（受控）： addType 在 T.type 已计算完之后、并且在无法回溯或取消的上下文中执行
- 非法情形（不受控）： 在存在回溯的解析器里（如带回溯的递归下降/PEG 回溯）直接执行 addType，若解析失败回溯，会导致符号表状态与解析状态不同步 → 需要回滚或缓冲

语法制导定义的基本概念

- 受控副作用的例子
- $L \rightarrow E \text{ n } \text{print}(E.\text{val})$
 - 通过副作用打印出E的值
 - 总是在最后执行，而且不会影响其它属性的求值
- 变量声明的SDD中的副作用
 - addType将标识符的类型信息加入到标识符表中
 - 只要标识符不被重复声明，标识符的类型信息总是正确的

产生式	语义规则
1) $D \rightarrow T L$	$L.inh = T.type$
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$
3) $T \rightarrow \text{float}$	$T.type = \text{float}$
4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $addType(\text{id.entry}, L.inh)$
5) $L \rightarrow \text{id}$	$addType(\text{id.entry}, L.inh)$

抽象语法树与类型

- 抽象语法树的构造

抽象语法树与类型

抽象语法树（Abstract Syntax Tree）

- 基本类型和数组类型的L属性定义

VS.

具象语法树（Concrete Syntax Tree）

抽象语法树与类型

- 具象语法树（Concrete Syntax Tree）：按照文法规则推导得到的完整树，精确反映文法的层次结构
 - 特点：包含所有非终结符与终结符；结构冗余，贴近文法；强调推导过程
- 抽象语法树（Abstract Syntax Tree）：去除了文法中冗余的语法细节，只保留与程序语义相关的核心结构
 - 特点：节点表示语言的核心成分（如“加法”、“变量”）；忽略不必要的非终结符（如括号层次、文法辅助符号）；更接近于编译器内部的中间表示。

抽象语法树与类型

特性	具体语法树（CST）	抽象语法树（AST）
目的	展示文法推导过程	表达程序语义的核心结构
节点内容	所有非终结符和终结符	语义相关的构造，如运算符、标识符
冗余性	高，结构庞大	精简，去掉无用信息
应用场景	理论教学，文法验证	编译器前端，中间代码生成

CST = 全部语法细节（就像考试解题步骤）
AST = 最终简化结果（就像答案公式）
在编译器中，AST 是后续 语义分析、优化和代码生成的核心输入

抽象语法树与类型

- 语法树（CST）：层次复杂，包含大量冗余符号。
- 抽象语法树（AST）：精简，直接反映程序的语义结构。
- 在 AST 上进行属性计算，效率更高，逻辑更清晰。
- 应用场景：
 - 类型检查（type checking）
 - 作用域/符号表管理
 - 常量折叠、优化
 - 中间代码生成

抽象语法树与类型

- 抽象语法树
 - 每个结点代表一个语法结构；对应于一个运算符
 - 结点的每个子结点代表其子结构；对应于运算分量
 - 表示这些子结构按照特定方式组成了较大的结构
 - 可以忽略掉一些标点符号等非本质的东西
- 构造抽象语法树的SDD
- 语法树的表示方法
 - 每个结点用一个对象表示
 - 对象有多个域
 - 叶子结点中只存放词法值
 - 内部结点中存放了op值和参数（通常指向其它结点）

抽象语法树与类型

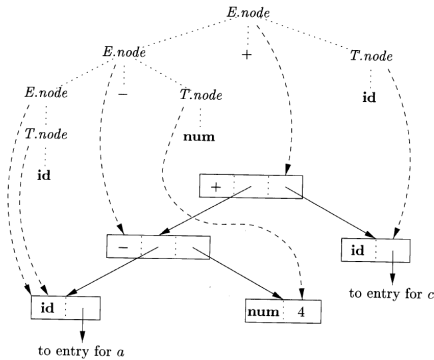
- 构造抽象语法树的SDD 的简单示例
- 属性E.node指向E对应的语法树的根结点

产生式	语义规则
1) $E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
2) $E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
3) $E \rightarrow T$	$E.node = T.node$
4) $T \rightarrow (E)$	$T.node = E.node$
5) $T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
6) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

抽象语法树与类型

- 构造抽象语法树的构造过程
- 示例1 输入：
a-4+c
- 步骤：
 - p1=new Leaf(id, entry_a)
 - p2=new Leaf(num, 4);
 - p3=new Node('-', p1,p2);
 - p4=new Leaf(id, entry_c);
 - p5=new Node('+', p3,p4);

产生式	语义规则
1) $E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
2) $E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
3) $E \rightarrow T$	$E.node = T.node$
4) $T \rightarrow (E)$	$T.node = E.node$
5) $T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
6) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

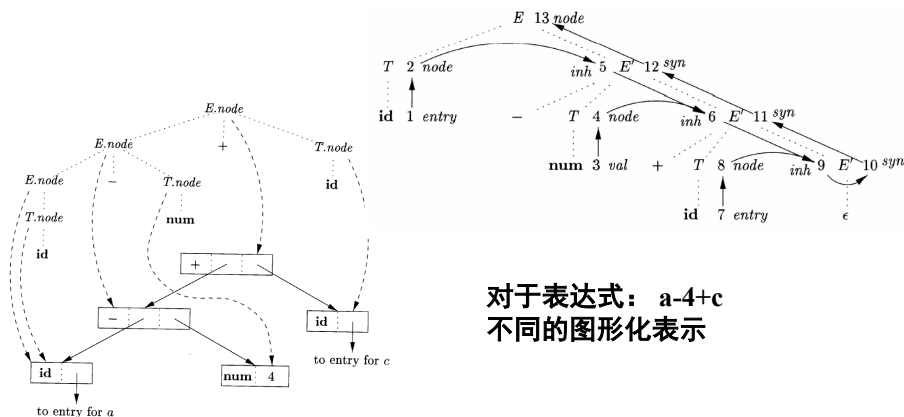


语法制导翻译的应用

- 在消除左递归时，按照规则得到此SDD

产生式	语义规则
1) $E \rightarrow T E'$	$E.node = E'.syn$ $E'.inh = T.node$
2) $E' \rightarrow + T E'_1$	$E'_1.inh = \text{new Node}('+', E'.inh, T.node)$ $E'.syn = E'_1.syn$
3) $E' \rightarrow - T E'_1$	$E'_1.inh = \text{new Node}('-', E'.inh, T.node)$ $E'.syn = E'_1.syn$
4) $E' \rightarrow \epsilon$	$E'.syn = E'.inh$
5) $T \rightarrow (E)$	$T.node = E.node$
6) $T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
7) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

语法制导翻译的应用

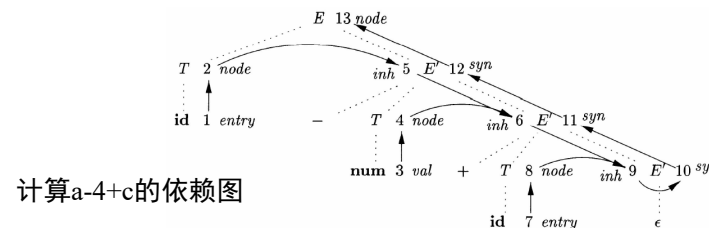


抽象语法树与类型

- 在 SDT 中语义动作不仅用于生成 AST，还可：
 - 在遇到声明语句 时：向符号表插入条目
 - 在遇到变量使用 时：查询符号表进行检查
- 在 AST 中每个标识符结点 都需要绑定符号表中的条目
- 符号表与 AST 协同工作：
 - AST 管理结构（树形层次）
 - 符号表管理名字（作用域与属性）

语法制导翻译的应用

- 对于这个SDD，各属性值的计算过程实际上和原来S属性定义中的计算过程一致
- 继承属性可以把值从一个结构传递到另一个并列的结构；也可把值从父结构传递到子结构
- 抽象语法树和分析树不一致时，继承属性很有用



抽象语法树与类型

- 构造抽象语法树的构造过程
- 示例2 输入：
 $\text{id} + \text{id} * \text{id}$

$$\begin{aligned} E \rightarrow E1 + T & \{ E.node = \text{Node}('+', E1.node, T.node) \} \\ E \rightarrow T & \{ E.node = T.node \} \\ T \rightarrow \text{id} & \{ T.node = \text{Node}(\text{"id"}, \text{id.lexeme}) \} \end{aligned}$$
- 步骤：
 - $T \rightarrow \text{id} \Rightarrow$ 生成 $\text{Node}(\text{"id"})$
 - $T \rightarrow \text{id} \Rightarrow$ 生成 $\text{Node}(\text{"id"})$
 - $T \rightarrow \text{id} \Rightarrow$ 生成 $\text{Node}(\text{"id"})$
 - $E \rightarrow T \Rightarrow$ 继承 $\text{Node}(\text{"id"})$
 - $E \rightarrow E + T \Rightarrow$ 生成 $\text{Node}('+', E1.node, T.node)$
 - $E \rightarrow E * T \Rightarrow$ 生成 $\text{Node}('*', E1.node, T.node)$

抽象语法树与类型

- 示例：


```
int x;
int f(int y) {
    return x + y;
}
```

- AST 负责表达式树形结构

- 符号表记录：x:int（全局），y:int（局部），f:function

- 符号表是语法树之外的“语义支撑系统”

抽象语法树与类型

- 符号表的基本概念
- 定义
 - 符号表是编译器中用于记录源程序中各种符号（变量、函数、类型等）的数据结构。
 - 本质上是一个 映射表（名称 → 属性）
- 作用
 - 在语义分析阶段，保存标识符的语义信息
 - 在后续的 中间代码生成、优化、目标代码生成 阶段提供支持。

抽象语法树与类型

- 符号表的基本概念
- 符号表的用途
 - 检查语义合法性
 - ❑ 变量是否声明后使用？
 - ❑ 类型是否匹配？
 - 作用域管理
 - ❑ 支持嵌套作用域（如函数、代码块）
 - 支持代码生成
 - ❑ 为每个变量分配存储空间或寄存器
 - 优化辅助
 - ❑ 确定变量的生命周期，进行寄存器分配或常量传播

抽象语法树与类型

- 符号表的作用：示例程序
 - 构建过程
 - 全局作用域
 - ❑ `x : int`
 - ❑ `f : function(int) -> int`
 - 函数 `f` 的局部作用域
 - ❑ `y : int`
 - ❑ `z : int`
- ```
int x; // 全局变量
int f(int y) {
 int z;
 z = x + y;
 return z;
}
```

# 抽象语法树与类型

- 符号表的基本概念
- 记录的信息
  - 符号名称（Identifier）
  - 类型信息（Type）
  - 作用域（Scope）
  - 存储分配（Memory Location）
  - 其他属性（如函数参数列表、是否为常量等）

# 抽象语法树与类型

- 符号表的基本概念
- 常见实现方式
  - 哈希表（Hash Table）：查找效率高（ $O(1)$ ）
  - 链表（Linked List）：适合作用域嵌套结构
  - 树（如平衡树）：适合需要有序遍历的情况
- 支持作用域的设计
  - 单表 + 作用域链：符号表栈（stack of tables）
  - 进入新作用域 → 新建符号表；退出作用域 → 弹出符号表

# 抽象语法树与类型

- 符号表的作用：示例程序
  - 组织结构
    - 全局符号表
      - ❑ `x : int`
      - ❑ `f : function(int) -> int`
    - `f` 的局部符号表
      - ❑ `y : int`
      - ❑ `z : int`
- ```
int x;           // 全局变量
int f(int y) {
    int z;
    z = x + y;
    return z;
}
```

- 链接到上层（全局作用域）
- 查找时：先查当前作用域 → 若未找到，则逐层向上查找

抽象语法树与类型

- 符号表在AST上的应用
- 抽象语法树的每个叶子结点（标识符）在语义分析时要绑定到符号表
- 符号表可作为属性文法的 继承属性 或 语义动作 的一部分：
 - 声明语句：在符号表中插入新条目
 - 使用语句：查符号表并进行类型检查

抽象语法树与类型

- 类型包括两个部分： $T \rightarrow B \ C$
 - 基本类型 B
 - 分量 C
- 分量形如 $[3][4]$
 - 表示3X4的二维数组
- $\text{int } [3][4]$
- 数组构造算符array
 - $\text{array}(3, \text{array}(4, \text{int}))$ 表示抽象的3X4的二维数组

产生式	语义规则
$T \rightarrow B \ C$	$T.t = C.t$ $C.b = B.t$
$B \rightarrow \text{int}$	$B.t = \text{integer}$
$B \rightarrow \text{float}$	$B.t = \text{float}$
$C \rightarrow [\text{num}] \ C_1$	$C.t = \text{array}(\text{num.val}, C_1.t)$ $C_1.b = C.b$
$C \rightarrow \epsilon$	$C.t = C.b$

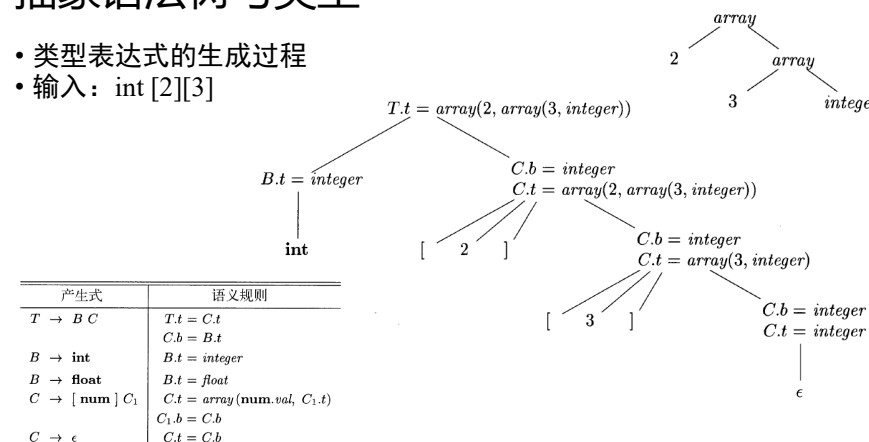
抽象语法树与类型

- 简化的类型表达式的语法
 - $T \rightarrow B \ C$
 - $B \rightarrow \text{int} \mid \text{float}$
 - $C \rightarrow [\text{num}] \ C \mid \epsilon$
- 生成类型表达式的SDD

产生式	语义规则
$T \rightarrow B \ C$	$T.t = C.t$ $C.b = B.t$
$B \rightarrow \text{int}$	$B.t = \text{integer}$
$B \rightarrow \text{float}$	$B.t = \text{float}$
$C \rightarrow [\text{num}] \ C_1$	$C.t = \text{array}(\text{num.val}, C_1.t)$ $C_1.b = C.b$
$C \rightarrow \epsilon$	$C.t = C.b$

抽象语法树与类型

- 类型表达式的生成过程
- 输入： $\text{int } [2][3]$



产生式	语义规则
$T \rightarrow B \ C$	$T.t = C.t$ $C.b = B.t$
$B \rightarrow \text{int}$	$B.t = \text{integer}$
$B \rightarrow \text{float}$	$B.t = \text{float}$
$C \rightarrow [\text{num}] \ C_1$	$C.t = \text{array}(\text{num.val}, C_1.t)$ $C_1.b = C.b$
$C \rightarrow \epsilon$	$C.t = C.b$

抽象语法树与类型

- 练习题：令 $S.val$ 为下面文法由 S 生成的二进制数的值（例如，对于输入101.101, $S.val=5.625$ ）：
 - $S \rightarrow L.L$
 - $S \rightarrow L$
 - $L \rightarrow LB$
 - $L \rightarrow B$
 - $B \rightarrow 0$
 - $B \rightarrow 1$

按语法制导翻译的方法，给出相应的语义规则。

抽象语法树与类型

- 练习题：令 $S.val$ 为下面文法由 S 生成的二进制数的值（例如，对于输入101.101, $S.val=5.625$ ）：

产生式	语义规则
$S \rightarrow S$	$\text{print}(S.val)$
$S \rightarrow L_1.L_2$	$S.val := L_1.val + L_2.val / 2^{L_2.length}$
$S \rightarrow L$	$S.val := L.val$
$L \rightarrow L_1B$	$L.val := L_1.val * 2 + B.val$ $L.length := L_1.length + 1$
$L \rightarrow B$	$L.val := B.val$ $L.length := 1$
$B \rightarrow 0$	$B.val := 0$
$B \rightarrow 1$	$B.val := 1$

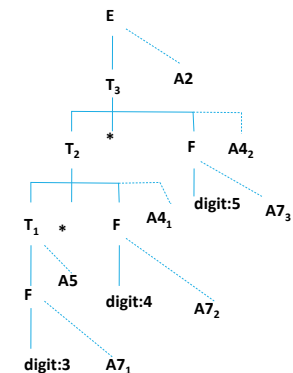
按语法制导翻译的方法，给出相应的语义规则。

语法制导翻译的应用

- 语法制导的翻译方案（SDT）是在产生式体中嵌入程序片断（语义动作）的上下文无关文法
- SDT的基本实现方法
 - 建立语法分析树
 - 将语义动作看作是虚拟的结点
 - 从左到右、深度优先地遍历分析树，在访问虚拟结点时执行相应动作

语法制导翻译的应用

L	$\rightarrow$	$E \mathbf{n}$	$\{ \text{print}(E.val); \}$	A_1
E	$\rightarrow$	$E_1 + T$	$\{ E.val = E_1.val + T.val; \}$	A_2
E	$\rightarrow$	T	$\{ E.val = T.val; \}$	A_3
T	$\rightarrow$	$T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$	A_4
T	$\rightarrow$	F	$\{ T.val = F.val; \}$	A_5
F	$\rightarrow$	(E)	$\{ F.val = E.val; \}$	A_6
F	$\rightarrow$	digit	$\{ F.val = \text{digit.lexval}; \}$	A_7



- 语句3*4*5的分析树如右
- DFS可知动作执行顺序
 - $A_7_1, A_5, A_7_2, A_4_1, A_7_3, A_4_2, A_2$
 - 注意，一个动作的不同实例所访问的属性值属于不同的结点

语法制导翻译的应用

- 实现SDT时，实际上并不会真的构造语法分析树，而是在分析过程中执行语义动作
- SDT可以看做是SDD的具体实施方案
- 即使基础文法可以应用某种分析技术，仍可能因为动作的缘故（**例如属性计算过程中的循环依赖**）导致此技术不可应用
- 用SDT实现两类重要的SDD
 - 基本文法可以使用LR分析技术，且SDD是S属性的
 - 基本文法可以使用LL分析技术，且SDD是L属性的

语法制导翻译的应用

- 判断是否可在分析过程中实现
- 具体示例：输入 $3 + 4 * 5$
 - 执行顺序：

$E \rightarrow E + T \quad \{ E.val = E_1.val + T.val \}$
 $E \rightarrow T \quad \{ E.val = T.val \}$
 $T \rightarrow T * F \quad \{ T.val = T_1.val * F.val \}$
 $T \rightarrow F \quad \{ T.val = F.val \}$
 $F \rightarrow (E) \quad \{ F.val = E.val \}$
 $F \rightarrow \text{id} \quad \{ F.val = \text{id.lexval} \}$

解析 $F \rightarrow \text{id} \{ F.val = \text{id.lexval} \}$ ，得 $F.val=3$
 上推 $T \rightarrow F \{ T.val = F.val \}$ ，得 $T.val=3$
 上推 $E \rightarrow T \{ E.val = T.val \}$ ，得 $E.val=3$
 解析 $F \rightarrow \text{id} \{ F.val = \text{id.lexval} \}$ ，得 $F.val=4$
 上推 $T \rightarrow F \{ T.val = 4 \}$
 解析 $F \rightarrow \text{id} \{ F.val = \text{id.lexval} \}$ ，得 $F.val=5$
 上推 $T \rightarrow T * F \{ T.val = 4 * 5 = 20 \}$
 最后 $E \rightarrow E + T \{ E.val = 3 + 20 = 23 \}$

语法制导翻译的应用

- 判断是否可在分析过程中实现
 - 将每个语义动作替换为一个独有的标记非终结符号；每个标记非终结符号M的产生式为 $M \rightarrow \epsilon$
 - 如果新的文法可以由某种方法进行分析，那么这个SDT就可以在这个分析过程中实现
 - 注意：这个方法没有考虑变量值的传递等要求

语法制导翻译的应用

- 将S属性的SDD转换为SDT
 - 非常简单，由于S属性的SDD中，所有的属性均为综合属性，因此，只有当所有子属性计算完毕之后，才能计算父节点
 - 因此，我们只需要**把所有的语义规则放到对应产生式的右部即可**

语法制导翻译的应用

- 将S属性的SDD转换为SDT
 - 非常简单，由于S属性的SDD中，所有的属性均为综合属性，因此，只有当所有子属性计算完毕之后，才能计算父节点
 - 因此，我们只需要**把所有的语义规则放到对应产生式的右部即可**
- 将L属性的SDD转换为SDT
 - 将计算某个**非终结符号A的继承属性的动作插入到产生式右部中紧靠在A的本次出现之前的位置上**；
 - 将计算一个产生式左部符号的综合属性的动作放置在这个产生式右部的最右端；

语法制导翻译的应用

- 如果一个L属性的SDD的基本文法可以使用LL分析技术，那么它的SDT可以在LL或者LR语法分析过程中实现；

$L \rightarrow E \mathbf{n}$	$\{ \text{print}(E.val); \}$	A_1		
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val; \}$	A_2		
$E \rightarrow T$	$\{ E.val = T.val; \}$	A_3		
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$	A_4		
$T \rightarrow F$	$\{ T.val = F.val; \}$	A_5		
$F \rightarrow (E)$	$\{ F.val = E.val; \}$	A_6		
$F \rightarrow \mathbf{digit}$	$\{ F.val = \mathbf{digit.lexval}; \}$	A_7		

$L \rightarrow E \mathbf{n}$	M_1	$M_1 \rightarrow \epsilon$
$E \rightarrow E+T$	M_2	$M_2 \rightarrow \epsilon$
$E \rightarrow T$	M_3	$M_3 \rightarrow \epsilon$
$\dots$	$\dots$	$\dots$

语法制导翻译的应用

- 后缀翻译方案
 - 后缀SDT：所有动作都在产生式最右端的SDT
 - 文法可以自底向上分析且SDD是S属性的，必然可以构造出后缀SDT
 - 构造方法
 - 将每个语义规则看作是一个赋值语义动作
 - 将所有的语义动作放在规则的最右端

语法制导翻译的应用

- 后缀翻译方案
 - 后缀SDT：所有动作都在产生式最右端的SDT
 - 文法可以自底向上分析且SDD是S属性的，必然可以构造出后缀SDT
 - 构造方法
 - 将每个语义规则看作是一个赋值语义动作
 - 将所有的语义动作放在规则的最右端

$L \rightarrow E \mathbf{n}$	$\{ \text{print}(E.val); \}$
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val; \}$
$E \rightarrow T$	$\{ E.val = T.val; \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$
$T \rightarrow F$	$\{ T.val = F.val; \}$
$F \rightarrow (E)$	$\{ F.val = E.val; \}$
$F \rightarrow \mathbf{digit}$	$\{ F.val = \mathbf{digit.lexval}; \}$

- 实现桌上计算器的后缀SDT
 - 注意动作中对属性值的引用
 - 我们允许语句引用全局变量，局部变量，文法符号的属性
 - 文法符号的属性只能被赋值一次

语法制导翻译的应用

- 基于栈的实现：
- 可以在LR语法分析的过程中实现
 - 归约时执行相应的语义动作
 - 定义用于记录各文法符号的属性的union结构
 - 栈中的每个文法符号（或者说状态）都附带一个这样的union类型的值
 - 在按照产生式 $A \rightarrow XYZ$ 归约时，Z的属性可以在栈顶找到，Y的属性可以在下一个位置找到，X的属性可以在再下一个位置找到

	X	Y	Z	状态 / 文法符号
	X.x	Y.y	Z.z	综合属性

↑
栈顶

语法制导翻译的应用

- 基于栈的实现：
- 假设语法分析栈存放在一个被称为stack的记录数组中，下标top指向栈顶
 - stack[top]是这个栈的栈顶
 - stack[top-1]指向栈顶下一个位置
 - 如果不同的文法符号有不同的属性集合，我们可以使用union来保存这些属性值
 - 归约时能够知道栈顶向下的各个符号分别是什么，因此我们也能够确定各个union中究竟存放了什么样的值

- 基于栈的实现：

产生式	语义动作
$L \rightarrow E \text{ n}$	{ $\text{print}(\text{stack}[\text{top} - 1].\text{val});$ $\text{top} = \text{top} - 1;$ }
$E \rightarrow E_1 + T$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} + \text{stack}[\text{top}].\text{val};$ $\text{top} = \text{top} - 2;$ }
$E \rightarrow T$	
$T \rightarrow T_1 * F$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} \times \text{stack}[\text{top}].\text{val};$ $\text{top} = \text{top} - 2;$ }
$T \rightarrow F$	
$F \rightarrow (E)$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 1].\text{val};$ $\text{top} = \text{top} - 2;$ }
$F \rightarrow \text{digit}$	

工业级编译器中的语义分析

- Javac流程细节
- 模块组织（源码在com.sun.tools.javac.comp包）：
 - Enter：符号表建立（类、方法、字段声明的引入与作用域分配）
 - MemberEnter：类体成员的详细处理，包括字段、方法、内部类等
 - Attr：类型检查和类型推断的主力模块
 - Check：约束性检查，如override、final等
 - Flow：流分析，包括变量初始化、到达性检查等

工业级编译器中的语义分析

- Javac （OpenJDK 的 Java 编译器）
- Javac 把编译分成若干明确的 pass
 - Parse
 - Enter（把声明进入符号表）
 - Annotation processing
 - Attribute (Attr)
 - Flow
 - Lower/TransTypes
 - Generate
- 语义相关的工作（名称解析、类型检查、常量折叠、数据流分析）主要在 com.sun.tools.javac.comp 包中的 Enter、Attr、Flow 等阶段完成

工业级编译器中的语义分析

- Javac流程细节
- 处理顺序：
 - 解析 -> 生成AST
 - Enter遍历AST，建立全局符号表（Scope）
 - MemberEnter填充类的成员符号
 - Attr按需递归遍历AST，做类型检查、推断
 - Check和Flow负责额外的语义规则和数据流分析
- 数据结构：
 - AST（JC树结构）：每种语言结构有对应的节点类型（如JCClassDecl、JCMethodDecl）
 - 符号（Symbol）：每个标识符有唯一的Symbol对象，携带类型、作用域、修饰符等信息
 - 作用域（Scope）：层级嵌套，符号表构成树状结构，支持局部/类/包/全局作用域
 - 类型系统（Type）：类型对象分为基本类型、类类型、泛型类型、数组类型、TypeVar等

工业级编译器中的语义分析

- Javac流程细节
- 关键机制
 - 泛型类型推断：支持类型擦除、桥方法、通配符推断，所有泛型相关语义都在Attr和Types模块中处理
 - 多态方法解析：方法查找、重载决议、访问权限检查、参数类型兼容性判断
 - 异常检查：静态检查throws、try-catch等
 - 数据流分析：如Flow模块对变量赋值到达性分析，检测未初始化变量和不可达语句
 - 错误/告警报告：通过Log和Diagnostic模块集中处理，支持精确定位
- SDT 的实践：
 - Javac中对SDT 进行了高度解耦：javac将语法、语义分离，语义全部在AST二次递归遍历中处理，便于维护和扩展

工业级编译器中的语义分析

- gcc（GNU C/C++ 编译器）
- 处理流程与模块分工
- 前端处理模块：
 - c-parser.c/cp/parser.c：词法和语法分析，生成parse tree
 - c-decl.c/cp/decl.c：声明和定义处理，符号表管理
 - c-typeck.c/cp/typeck.c：类型检查和相关语义分析
 - c-semantics.c/cp/semantics.c：语义动作和上下文敏感处理
- 流程：
 - 生成parse tree（AST）
 - 多轮遍历处理声明、类型和作用域
 - 语义分析期间生成GIMPLE（统一中间表示）
 - 进一步分析如模板实例化、重载决议等

工业级编译器中的语义分析

- gcc (GNU C/C++ 编译器)
- 数据结构
 - tree节点: 所有语法和语义结构都为 “tree”对象 (共用tag/union结构)
 - symbol_table: 多级嵌套哈希表, 存储变量、类型、标签、函数等符号信息
 - type结构: 包含原生类型、结构体、联合体、指针、数组、函数类型等

工业级编译器中的语义分析

- gcc (GNU C/C++ 编译器)
- SDT 的实践:
 - 传统SDT嵌入: 早期GCC大量采用Bison语法文件中的动作代码, 许多语义分析在归约时完成
 - 工程化后移: 近年来逐步将复杂语义分析抽出到专门的语义分析模块, 减少语法动作臃肿

工业级编译器中的语义分析

- rustc (Rust 编译器)
- 处理流程与模块分工
- 分析顺序:
 - 解析AST, 扩展宏, 生成HIR
 - 名称解析+作用域分析
 - 类型推断与类型检查
 - 借用和生命周期分析
 - 生成MIR (中间代码)

工业级编译器中的语义分析

- gcc (GNU C/C++ 编译器)
- 关键机制
 - C++模板与重载决议:
 - ❑ 模板参数推断与实例化通过延迟绑定和多轮分析完成
 - ❑ 函数重载、运算符重载在归约阶段根据符号表和类型信息决议
 - 类型兼容与转换:
 - ❑ 支持隐式/显示类型转换, 自动插入转换节点
 - ❑ 检查const、volatile、restrict等限定符兼容性
 - 声明提前/多重定义检测:
 - ❑ 支持C语言中的声明提前和多重定义, 符号表及时更新和错误报告
 - ❑ C++特有分析: 虚函数表、继承层次、访问权限、命名空间等
 - 作用域管理:
 - ❑ 每个block、函数、结构体都维护单独的符号表, 出入作用域时动态切换

工业级编译器中的语义分析

- rustc (Rust 编译器)
- 处理流程与模块分工
- 主要模块:
 - rustc_parse: 生成AST
 - rustc_hir: AST转HIR (高层中间表示), 适合语义分析
 - rustc_resolve: 名称解析、模块路径、宏展开
 - rustc_typeck: 类型检查与推断
 - rustc_borrowck: 所有权、生命周期和借用检查
 - rustc_lint: 静态检查和代码风格建议

工业级编译器中的语义分析

- rustc (Rust 编译器)
- 数据结构
 - HIR (High-level IR): 比AST更加简洁, 去除了宏和语法糖, 便于多遍分析
 - DefId/NodeId系统: 唯一标识每个定义或节点, 方便跨模块引用
 - 符号表 (NameResolution): 多级嵌套哈希表, 支持模块、impl、trait、泛型参数等
 - 类型系统: 包含通用类型、trait对象、泛型、生命周期标记 (Region)、所有权等

工业级编译器中的语义分析

- rustc (Rust 编译器)
- 关键机制
 - 名称解析：
 - ❑ 支持复杂的模块系统、宏系统 (含宏定义和展开时的作用域隔离)
 - ❑ 完成后保证所有标识符唯一绑定
 - 类型推断：
 - ❑ Hindley-Milner 风格, 支持闭包、泛型、trait bound 自动推断
 - ❑ 支持 “部分推断失败” 时的错误提示与建议
 - 所有权/生命周期/借用检查：
 - ❑ 所有权系统通过数据流和控制流联合分析
 - ❑ 生命周期通过 “region inferencer” 全程序推断
 - ❑ 借用检查器 (NLL/Polonius) 构建借用关系图, 分析生命周期冲突
 - trait/impl 分析：
 - ❑ trait bound 约束、impl block 一致性检查、自动推导和重叠检测
 - 错误与建议：
 - ❑ 高级错误信息与自动修复建议, 部分分析器支持 “次优解” 建议

工业级编译器中的语义分析

工业级编译器中的语义分析

- rustc (Rust 编译器)
- SDT 风格
 - 完全解耦: 所有语义分析通过多轮遍历HIR、MIR, 不和语法规则直接绑定
 - 工程化注解: 分析过程中不断为HIR/MIR节点注解类型、生命周期等信息, 便于后续依赖

编译器	语义分析触发点	数据结构/表示	多遍分析	典型难点	SDT实现风格
javac	AST遍历阶段	Symbol/Scope/Type	多遍	泛型推断、多文件依赖	高度解耦, 面向对象
gcc	归约/多遍遍历	tree/符号表	多遍	模板、重载、多语言支持	传统SDT+后期分离
rustc	HIR/MIR多遍遍历	HIR/DefId/Region	多遍	所有权/生命周期/宏系统	完全解耦, 注解式

第三章 语义分析 (属性文法)

冯洋
南京大学计算机学院
fengyang@nju.edu.cn